



Day 32 — React Event Handling

Contact Form with Real-Time Validation

Date: January 16, 2026 (Thursday) | **Phase 2, Week 5**



Mission: By end of today, you will master how React responds to user actions — understanding the bridge between user interactions and state updates that power every modern web app.



MAX OMEGA PRIME INTEGRITY CHECK

Before reading further — answer these NOW:

1. ☒ Was your Integrity Session completed today (7:30 AM – 10:30 AM)?
2. ☒ Do you have Day 31 proof? (ToDo List committed to GitHub?)
3. ☒ Can you explain: *What happens when you call `setCount(count + 1)`?*

If any answer is NO — stop. Yesterday's useState must be crystal clear.

PART 1: CONCEPT FOUNDATION

Building the RIGHT Mental Model First



The Video Game Controller Analogy — Understanding Events

Imagine playing Cricket 19 on your PlayStation at home in Lahore. You press buttons:



CONTROLLER BUTTONS:



= Shot selection



= Run between wickets



= Jump/dive



= Pause menu

When you press  → Game DETECTS the press → Game EXECUTES shot animation

This is EXACTLY how React event handling works:

```
<button onClick={hitSix}>Hit Six!  </button>
```



Event Type (like ✕)	Function to Execute (like "shot animation")
------------------------	--

The flow:

1. User interacts with page (click, type, submit)
2. Browser detects the interaction (creates an event)
3. React calls your function
4. Your function updates state
5. React re-renders with new state

Key insight: Events are the **bridge** between user actions and state changes.

The Phone Call Analogy — Event Handlers

Think about receiving a phone call in Pakistan:

INCOMING CALL FLOW:

1. Phone rings (EVENT happens)
2. You see caller ID (EVENT data)
3. You decide: Answer or Ignore (HANDLER function)
4. If answer: Conversation starts (HANDLER executes)

Your phone has a "ring handler" – you!

In React:

```
function ContactForm() {  
  // This is your "ring handler" for button clicks  
  function handleSubmit() {  
    console.log("Form submitted!");  
  }  
  
  return <button onClick={handleSubmit}>Submit</button>;  
  //           ↑           ↑  
  //       Event type   Your handler  
}
```

Important naming:

- Event prop: `onClick`, `onChange` (camelCase, not lowercase)
- Handler function: `handleClick`, `handleChange` (convention: start with "handle")

The Careem Driver Analogy — Synthetic Events

When you book a Careem in Lahore, you don't deal directly with the driver's car mechanics. Careem provides a **consistent interface**:

YOU → Careem App → Driver's Car
(Abstraction)

- Press "Book" → Careem handles details → Car arrives
- Different cars (Honda, Suzuki, Toyota)
- But same interface for you!

React Synthetic Events work the same way:

USER → React Event → Browser Event
(Synthetic) (Native)

- Click on Chrome → React normalizes → Your handler receives consistent event
- Click on Firefox → React normalizes → Same event object!
- You write code ONCE, works everywhere

Why this matters:

- Different browsers have slight differences in events
- React smooths them out
- You get consistent **event** object across all browsers

Pause & Think Moment #1

Before reading further, answer this:

"In your Day 31 ToDo app, you wrote `onClick={() => handleDeleteTask(task.id)}`. Why the arrow function? Why not just `onClick={handleDeleteTask}`?"

Write your answer. This is the #1 confusion point for beginners.

Answer to verify:

- `onClick={handleDeleteTask}` calls the function on render (wrong!)
- `onClick={() => handleDeleteTask(task.id)}` creates a function that will be called later with the parameter (correct!)

We'll dive deeper into this today.

The Form Filling Analogy — Controlled Components

Think about filling a NADRA form for your CNIC in Pakistan:

📄 TRADITIONAL FORM (Uncontrolled):

- You write in the boxes
- NADRA reads it only when you submit
- They don't know what you wrote until submission

📄 REAL-TIME FORM (Controlled):

- You write in boxes
- NADRA assistant reads AS YOU WRITE
- They validate instantly
- They can guide you in real-time

React Controlled Components = Real-time form:

```
// CONTROLLED: React "knows" input value at all times
function NameInput() {
  const [name, setName] = useState("");

  return (
    <input
      value={name} // React controls the value
      onChange={(e) => setName(e.target.value)} // Updates state on change
    />
  );
}

// React always knows: name = "whatever user typed"
// Can validate immediately, format it, prevent certain characters, etc.
```

Uncontrolled (old way, avoid in React):

```
// UNCONTROLLED: React doesn't track value
function NameInput() {
  return <input />; // Browser controls value, React has no idea what's in it
}
```

The key difference:

- **Controlled:** State → Input value (Single source of truth)
- **Uncontrolled:** Input manages its own value (React doesn't know)

PART 2: FUNDAMENTAL BUILDING BLOCKS

One Concept at a Time

Building Block #1 — Basic Event Handlers (onClick)

The simplest form:

```
function ClickButton() {  
  function handleClick() {  
    console.log("Button clicked!");  
  }  
  
  return <button onClick={handleClick}>Click Me</button>;  
  //           ↑ Pass function REFERENCE, not call it  
}
```

Three ways to write handlers (all valid):

```
// Method 1: Separate named function (RECOMMENDED for clarity)  
function handleClick() {  
  console.log("Clicked!");  
}  
<button onClick={handleClick}>Click</button>  
  
// Method 2: Inline arrow function  
<button onClick={() => console.log("Clicked!")}>Click</button>  
  
// Method 3: Inline function expression  
<button onClick={function() { console.log("Clicked!"); }}>Click</button>
```

Common Mistake #1:

```
// ❌ WRONG: Calls function immediately on render  
<button onClick={handleClick()}>Click</button>  
// This executes handleClick RIGHT NOW, not on click  
  
// ✅ CORRECT: Passes function to be called later  
<button onClick={handleClick}>Click</button>
```

Building Block #2 — Event Object

Every event handler receives an **event object** with info about the event:

```
function ButtonWithEvent() {  
  function handleClick(event) {  
    console.log("Event type:", event.type);    // "click"  
  }  
}
```

```

    console.log("Target element:", event.target); // <button>
    console.log("Timestamp:", event.timeStamp);   // When click happened
  }

  return <button onClick={handleClick}>Click</button>;
}

```

Useful event properties:

Property	What it tells you	Example
<code>event.type</code>	Type of event	"click", "change", "submit"
<code>event.target</code>	Element that triggered event	<code><input></code> element
<code>event.target.value</code>	Input's current value	"Ahmed"
<code>event.target.checked</code>	Checkbox state	true/false
<code>event.preventDefault()</code>	Stop default behavior	Prevent form submission

Building Block #3 — onChange for Inputs

The controlled input pattern:

```

function NameInput() {
  const [name, setName] = useState("");

  function handleChange(event) {
    // event.target = the input element
    // event.target.value = what user typed
    setName(event.target.value);
  }

  return (
    <input
      type="text"
      value={name}           // State controls the input
      onChange={handleChange} // Update state on every keystroke
    />
  );
}

```

Shorter version (common in real code):

```

<input
  value={name}
  onChange={(e) => setName(e.target.value)}

```

```
// e = event, e.target.value = typed text
/>
```

Why `onChange` not `onInput`?

React uses `onChange` for consistency, but it fires on every keystroke (like HTML's `oninput`).

Building Block #4 — Passing Parameters to Handlers

The problem: You need to pass extra data to your handler.

```
// Example: Delete button for each task
function TaskList() {
  const [tasks, setTasks] = useState([
    { id: 1, text: "Buy groceries" },
    { id: 2, text: "Pay bills" }
  ]);

  function handleDelete(taskId) {
    // Need to know WHICH task to delete
    setTasks(tasks.filter(t => t.id !== taskId));
  }

  return (
    <div>
      {tasks.map(task => (
        <div key={task.id}>
          <span>{task.text}</span>

          {/* ❌ WRONG: Calls immediately */}
          <button onClick={handleDelete(task.id)}>Delete</button>

          {/* ✅ CORRECT: Wrap in arrow function */}
          <button onClick={() => handleDelete(task.id)}>Delete</button>
        </div>
      ))}
    </div>
  );
}
```

Why the arrow function works:

```
onClick={() => handleDelete(task.id)}

// This creates a NEW function: () => handleDelete(1)
// React calls this function when clicked
// Then THAT function calls handleDelete(1)
```

Alternative approach (less common):

```
<button onClick={(e) => handleDelete(task.id, e)}>Delete</button>  
// Can also receive event object if needed
```



Building Block #5 — onSubmit for Forms

Form submission has special behavior:

```
function ContactForm() {  
  const [name, setName] = useState("");  
  
  function handleSubmit(event) {  
    event.preventDefault(); // CRITICAL: Stop page refresh!  
  
    // Now process the form  
    console.log("Submitting:", name);  
    // Send to API, etc.  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input  
        value={name}  
        onChange={(e) => setName(e.target.value)}  
      />  
      <button type="submit">Submit</button>  
    </form>  
  );  
}
```

Why `event.preventDefault()`?

- By default, form submission RELOADS the page
- In React SPAs, we don't want page reload
- `preventDefault()` stops the default behavior
- Then we handle submission with JavaScript



Common Mistakes — Learn From Others' Pain

Mistake 1: Calling function instead of passing it

```
// ❌ Function executes on render  
<button onClick={console.log("Hi")}>Click</button>  
// Logs "Hi" immediately when component renders!
```



```
//  Function executes on click  
<button onClick={() => console.log("Hi")}>Click</button>
```

Mistake 2: Forgetting event.preventDefault() on forms

```
//  Page refreshes on submit  
function handleSubmit(event) {  
  console.log("Submitting..."); // Page reloads before this finishes  
}  
  
//  Prevents reload  
function handleSubmit(event) {  
  event.preventDefault(); // MUST be first line  
  console.log("Submitting..."); // Now this works  
}
```

Mistake 3: Not using arrow function when passing parameters

```
//  Calls deleteTask(1) immediately  
<button onClick={deleteTask(task.id)}>Delete</button>  
  
//  Creates function to call later  
<button onClick={() => deleteTask(task.id)}>Delete</button>
```

Mistake 4: Wrong event name casing

```
//  Lowercase (HTML style) – doesn't work in React  
<button onclick={handleClick}>Click</button>  
  
//  CamelCase (React style)  
<button onClick={handleClick}>Click</button>
```

PART 3: PROGRESSIVE LEARNING PATH

I Do → We Do → You Do



Stage 1 — I DO: Study This Working Example

Complete Form with Multiple Event Types

```
import { useState } from 'react';

function UserForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [agreed, setAgreed] = useState(false);

  // Handler for text inputs
  function handleNameChange(event) {
    setName(event.target.value);
  }

  // Handler for email input
  function handleEmailChange(event) {
    setEmail(event.target.value);
  }

  // Handler for checkbox
  function handleCheckboxChange(event) {
    setAgreed(event.target.checked); // Note: .checked not .value
  }

  // Handler for form submission
  function handleSubmit(event) {
    event.preventDefault(); // Stop page reload

    console.log("Form data:", { name, email, agreed });

    // Clear form after submit
    setName("");
    setEmail("");
    setAgreed(false);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Name"
        value={name}
        onChange={handleNameChange}
      />

      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={handleEmailChange}
      />

      <label>
        <input
```

```

        type="checkbox"
        checked={agreed} // For checkbox, use "checked" not "value"
        onChange={handleCheckboxChange}
      />
      I agree to terms
    </label>

    <button type="submit">Submit</button>
  </form>
);
}

```

🧠 Pause & Think Moment #2:

- Why does checkbox use `checked={agreed}` instead of `value={agreed}`?
- What happens if you remove `event.preventDefault()`?
- Why clear the form after submission?

Answer before proceeding.



Stage 2 — WE DO: Complete the Gaps Together

Task: Build a LoginForm with validation feedback

```

import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event) {
    // TODO 1: Prevent default form behavior
    _____;

    // TODO 2: Clear any previous errors
    setError(_____);

    // TODO 3: Validate email contains @ symbol
    if (!email.includes(_____)) {
      setError("Invalid email format");
      return; // Stop here if invalid
    }

    // TODO 4: Validate password is at least 6 characters
    if (password.length < _____) {
      setError("Password must be at least 6 characters");
      return;
    }
  }
}

```

```

    // If we reach here, validation passed
    console.log("Login successful:", { email, password });
  }

  return (
    <form onSubmit={_____}>
      {/* TODO 5: Show error message if error exists */}
      {_____} && <p style={{color: 'red'}}>{_____}</p>

      <input
        type="email"
        placeholder="Email"
        value={email}
        // TODO 6: Add onChange handler to update email state
        onChange={(e) => _____(_____)}
      />

      <input
        type="password"
        placeholder="Password"
        value={password}
        // TODO 7: Add onChange handler to update password state
        onChange={_____}
      />

      <button type="submit">Login</button>
    </form>
  );
}

```

Hints (read only if stuck 5+ minutes):

- Hint 1: `event.preventDefault()` to stop reload
- Hint 2: Clear error: `setError("")`
- Hint 3: Email check: `email.includes("@")`
- Hint 4: Password length: `password.length < 6`

Stage 3 — YOU DO: Independent Practice Problems

Tier 1 — Basic (10 minutes)

Build a **ColorPicker** component

Requirements:

- Three buttons: Red, Green, Blue
- Clicking any button changes text color
- Show "Current color: [color name]" text
- The text showing current color should be IN that color

Thinking Framework:

1. What state do I need? (color)
2. What event do I listen for? (onClick)
3. What should handler do? (setColor to clicked color)
4. How to apply color? (inline style)

🟡 Tier 2 — Intermediate (15 minutes)

Build a **SearchFilter** component (like Daraz search)

Requirements:

- Input field for search text
- State: **searchText**
- List of products (hardcoded array)
- Show only products that match search text (case-insensitive)
- Show "No results" if no match

Products array:

```
const products = [  
  "Samsung TV",  
  "iPhone 14",  
  "Laptop HP",  
  "Headphones Sony"  
];
```

Thinking Framework:

1. State needed: searchText
2. Filter logic: `products.filter(p => p.toLowerCase().includes(searchText.toLowerCase()))`
3. onChange: update searchText
4. Conditional rendering: Show list or "No results"

🔴 Tier 3 — Challenge (20 minutes)

Build a **RegistrationForm** with multi-field validation

Requirements:

- Fields: Name, Email, Phone, Password, Confirm Password

- Real-time validation for each field:
 - Name: Not empty
 - Email: Contains @ and .
 - Phone: Starts with 03 (Pakistani mobile)
 - Password: At least 8 characters
 - Confirm: Matches password
- Show error message below each invalid field
- Submit button disabled until ALL fields valid

This is complex. Break it down:

1. Five state variables for inputs
2. Five state variables for errors (or one object)
3. Validation function for each field
4. onChange handlers trigger validation
5. Submit button: disabled={hasAnyError}

Hint for disabled button:

```
const isFormValid =
  name !== "" &&
  email.includes("@") &&
  email.includes(".") &&
  phone.startsWith("03") &&
  password.length >= 8 &&
  confirmPassword === password;

<button disabled={!isFormValid}>Register</button>
```

Debugging Challenge

This form has 6 bugs. Find them WITHOUT running code:

```
function BuggyForm() {
  const [name, setName] = useState("");

  function handleChange(event) {
    setName(name);
  }

  function handleSubmit() {
    console.log("Submitted:", name);
  }

  return (
```

```
<form onsubmit={handleSubmit}>
  <input
    type="text"
    value={name}
    onChange={handleChange}
  />
  <button onclick={handleSubmit()}>Submit</button>
</form>
);
}
```

Bugs to find:

1. Bug with form's onSubmit
2. Bug with input's onChange
3. Bug with button's onClick
4. Bug with handleChange logic
5. Bug with handleSubmit parameters
6. Bug with button in form (hint: what's button's default type?)

Write down all 6 bugs before checking answer key.

PART 4: INDEPENDENT APPLICATION — PROJECT



Contact Form with Real-Time Validation



Your Mission

Build a **professional contact form** like you'd see on a company website, with instant validation feedback.

You are NOT given the complete solution. You must architect this yourself.



Project Requirements

Core Features (MUST HAVE):

1.  **Four Input Fields:**
 - Full Name (text)
 - Email Address (email)
 - Phone Number (tel)
 - Message (textarea)

2. **Real-Time Validation:**

- Validate as user types (onChange)
- Show  green checkmark if field valid
- Show  red X with error message if invalid
- Validation happens immediately, not on submit

3. **Validation Rules:**

- **Name:** Not empty, at least 3 characters
- **Email:** Contains @ and . in correct positions
- **Phone:** Pakistani format 03XX-XXXXXXX (11 digits)
- **Message:** At least 10 characters

4. **Submit Button:**

- Disabled if ANY field invalid
- Enabled only when ALL fields valid
- On submit: Show success message, clear form

5. **Error Messages:**

- Specific error for each validation failure
- "Name must be at least 3 characters"
- "Invalid email format"
- "Phone must be 11 digits starting with 03"
- "Message too short (minimum 10 characters)"

Implementation Roadmap (Build in This Order!)

Milestone 1 — Basic Form Structure (20 minutes)

What you need:

- Four state variables for input values
- Four controlled input fields
- Form with onSubmit handler
- Basic layout

Starter structure:

```
function ContactForm() {  
  const [name, setName] = useState("");  
  const [email, setEmail] = useState("");  
  const [phone, setPhone] = useState("");  
  const [message, setMessage] = useState("");  
  
  function handleSubmit(event) {  
    // TODO: Prevent default
```



```

    // TODO: Log form data
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* TODO: Add 4 controlled inputs */}
      <button type="submit">Send Message</button>
    </form>
  );
}

```

Success criteria:

- Can type in all fields
- Form doesn't reload page on submit
- Console logs all field values

Milestone 2 — Validation Functions (30 minutes)

What you need:

- Separate validation function for each field
- Each returns true/false or error message
- Functions are pure (no side effects)

Validation function examples:

```

// Returns empty string if valid, error message if invalid
function validateName(name) {
  if (name.length === 0) return "Name is required";
  if (name.length < 3) return "Name must be at least 3 characters";
  return ""; // Valid
}

function validateEmail(email) {
  // TODO: Check for @ symbol
  // TODO: Check for . after @
  // TODO: Check text before @, between @ and ., and after .
  return ""; // or error message
}

function validatePhone(phone) {
  // TODO: Remove spaces and dashes first
  // TODO: Check if 11 digits
  // TODO: Check if starts with 03
  return "";
}

function validateMessage(message) {

```

```
// TODO: Check minimum 10 characters  
return "";  
}
```

Success criteria:

- Each function returns "" when input valid
 - Each function returns specific error when invalid
 - Can test functions independently
-

Milestone 3 — Error State Management (20 minutes)

What you need:

- State for each field's error message
- Update errors onChange
- Display errors below each field

```
const [nameError, setNameError] = useState("");  
const [emailError, setEmailError] = useState("");  
const [phoneError, setPhoneError] = useState("");  
const [messageError, setMessageError] = useState("");  
  
function handleNameChange(event) {  
  const value = event.target.value;  
  setName(value);  
  
  // Validate immediately  
  const error = validateName(value);  
  setNameError(error);  
}  
  
// In JSX:  
<input  
  value={name}  
  onChange={handleNameChange}  
>  
{nameError && <span style={{color: 'red'}}>{nameError}</span>}
```

Success criteria:

- Error appears immediately when typing invalid input
 - Error disappears when input becomes valid
 - Each field has independent error state
-

Milestone 4 — Visual Validation Feedback (15 minutes)

What you need:

- Show  when field valid
- Show  when field invalid
- Color-code input border (green/red)

```
// Determine if field is valid
const isValidName = name.length >= 3;

<div>
  <input
    value={name}
    onChange={handleNameChange}
    style={{
      borderColor: nameError ? 'red' :
                    isValidName ? 'green' : 'gray'
    }}
  />
  {isValidName && <span> {nameError}</span>}
</div>
```

Success criteria:

- Green border + checkmark when valid
- Red border + error when invalid
- Gray border when empty (neutral state)

Milestone 5 — Smart Submit Button (15 minutes)

What you need:

- Check if ALL fields are valid
- Disable button if any invalid
- Show success message on submit
- Clear form after submit

```
// Check overall form validity
const isValidForm =
  validateName(name) === "" &&
  validateEmail(email) === "" &&
  validatePhone(phone) === "" &&
  validateMessage(message) === "";

function handleSubmit(event) {
  event.preventDefault();

  // Double-check validity (defensive)
```

```

    if (!isFormValid) return;

    // Show success
    alert("Message sent successfully!");

    // Clear form
    setName("");
    setEmail("");
    setPhone("");
    setMessage("");

    // Clear errors
    setNameError("");
    setEmailError("");
    setPhoneError("");
    setMessageError("");
}

<button type="submit" disabled={!isFormValid}>
  Send Message
</button>

```

Success criteria:

- Button disabled when any field invalid
- Button enabled when all fields valid
- Form clears after successful submit
- Success message displays

Thinking Frameworks

Framework 1: Email Validation Logic

Email structure: username@domain.extension

Checks needed:

1. Contains exactly one @
2. Text before @ (username)
3. Text after @ (domain)
4. Contains . after @
5. Text after . (extension like com, pk)

Implementation:

- Split by @ → should get 2 parts
- parts[0] must have length > 0
- parts[1] must contain .
- Split parts[1] by . → check both parts non-empty

Framework 2: Pakistan Phone Validation

Format: 03XX-XXXXXXX or 03XXXXXXXXX
Total: 11 digits starting with 03

Steps:

1. Remove all spaces and dashes: `phone.replace(/[- ]/g, "")`
2. Check if exactly 11 characters
3. Check if all are digits: `/^\d+$/ .test(cleaned)`
4. Check if starts with "03"

Framework 3: Real-Time vs Submit Validation

Real-time (onChange):

- Validate as user types
- Immediate feedback
- Better UX
- Prevents invalid submission

Submit validation:

- Validate only on submit
- Less annoying during typing
- Use as backup check

Best: Combine both!

- Real-time for feedback
- Submit validation as safety net

Self-Assessment Checklist

Before calling project "done", verify:

- ☐ All four fields are controlled components
- ☐ Validation happens on every keystroke (onChange)
- ☐ Error messages are specific and helpful
- ☐ Visual feedback (colors, icons) is clear
- ☐ Submit button is disabled when form invalid
- ☐ Submit button is enabled when form valid
- ☐ Form clears after successful submit
- ☐ No console errors
- ☐ `event.preventDefault()` prevents page reload
- ☐ Can explain the difference between controlled and uncontrolled
- ☐ Code committed to GitHub

Apply to New Context Questions

1. **Debouncing:** Real-time validation on every keystroke can be expensive. How would you validate only after user stops typing for 500ms?
 2. **Server-side validation:** Email uniqueness can't be checked client-side. How would you check if email exists in database as user types?
 3. **Multi-step forms:** How would you modify this to be a 3-step wizard (Personal Info → Contact → Message)?
 4. **File uploads:** How would you add an "Attach File" button using `<input type="file">`? What event would you listen to?
-



Extension Challenges (If Time Remains)

Challenge 1: Password Strength Meter

- Add password field with confirm password
- Show strength (Weak/Medium/Strong) with colored bar
- Requirements: 8+ chars, uppercase, lowercase, number, symbol

Challenge 2: Character Counter

- Show "X / 200 characters" below message field
- Turn red when exceeding 200
- Prevent typing beyond limit

Challenge 3: Form Analytics

- Track which field user focuses on first
 - Track time spent on each field
 - Log to console on submit
-



ORAL GATEKEEPING EXAM PREP

MAX OMEGA PRIME Will Ask You These

Level 1 — Remember (Concepts)

- What is a synthetic event?
- What's the difference between `onClick` and `onclick`?
- What does `event.preventDefault()` do?

Level 2 — Understand (Explain)

Using the Careem analogy, explain:

- Why React wraps browser events (synthetic events)

- What is a controlled component?
- Why do we need onChange for inputs in React?

Level 3 — Apply (Code Challenge — 10 minutes)

Build a **TemperatureConverter** (Celsius ↔ Fahrenheit):

- Two inputs: Celsius and Fahrenheit
- Typing in one auto-updates the other
- Both are controlled components
- Formulas:
 - $F = (C \times 9/5) + 32$
 - $C = (F - 32) \times 5/9$

Level 4 — Analyze (Debug)

Why doesn't this input work?

```
function BrokenInput() {  
  const [text, setText] = useState("");  
  return <input value={text} />;  
}
```

Explain what's wrong and what happens when user tries to type.

Level 5 — Teach Back

"Explain event handling in React to a friend who knows vanilla JavaScript but has never seen React. Use the Cricket game controller analogy."



Debugging Challenge — Answer Key

(Read ONLY after attempting yourself)

Original buggy code:

```
function BuggyForm() {  
  const [name, setName] = useState("");  
  
  function handleChange(event) {  
    setName(name); // Bug 4  
  }  
  
  function handleSubmit() { // Bug 5  
    console.log("Submitted:", name);  
  }  
}
```

```

return (
  <form onSubmit={handleSubmit}> {/* Bug 1 */}
    <input
      type="text"
      value={name}
      onChange={handleChange} {/* Bug 2 */}
    />
    <button onClick={handleSubmit()}>Submit</button> {/* Bug 3 */}
  </form>
);
}

```

Fixes:

```

function FixedForm() {
  const [name, setName] = useState("");

  function handleChange(event) {
    setName(event.target.value); // Fix 4: Use event.target.value, not name
  }

  function handleSubmit(event) { // Fix 5: Accept event parameter
    event.preventDefault(); // Fix 5b: Prevent page reload
    console.log("Submitted:", name);
  }

  return (
    <form onSubmit={handleSubmit}> {/* Fix 1: CamelCase onSubmit */}
      <input
        type="text"
        value={name}
        onChange={handleChange} {/* Fix 2: CamelCase onChange */}
      />
      <button type="button" onClick={handleSubmit}> {/* Fix 3: onClick, no
() */}
        Submit
      </button>
      {/* Fix 6: Button in form defaults to type="submit" */}
      {/* Either use type="button" with onClick */}
      {/* OR remove onClick and let form onSubmit handle it */}
    </form>
  );
}

```

Explanation of Bug 6:

Buttons inside `<form>` default to `type="submit"`. This button both:

- Triggers `onClick` (which calls `handleSubmit`)
- Submits the form (which also calls `handleSubmit`)

- Result: handleSubmit runs TWICE!

Better approach:

```
<form onSubmit={handleSubmit}>
  { /* ... inputs ... */ }
  <button type="submit">Submit</button>
  { /* Let form handle submission, remove onClick from button */ }
</form>
```

Tomorrow Preview — Day 33

Topic: Lists & Keys in React (Rendering Arrays Correctly)

You'll master:

- The .map() method in JSX
- Why keys are critical
- Avoiding index as key
- Conditional rendering patterns
- Performance optimization for lists

Tonight's prep question:

*"In your ToDo app and Contact Form, you rendered lists of items. Did you notice the **key** prop? Why is it needed? What happens without it?"*

Open DevTools console tomorrow — you might see warnings about missing keys if you forgot them!

MAX OMEGA PRIME's Closing Message

Sharjeel, mubarak ho! Aaj tu ne React ki **interaction layer** master kar li!

Ab teri applications sirf data dikhati nahi — **users ke saath baat karti hain:**

- User types → onChange fires → State updates → UI changes
- User clicks → onClick fires → Action happens → State updates → UI reflects
- User submits → onSubmit fires → Form processed → Success message → Form clears

Yeh React ka asli magic hai. Static pages se interactive apps ban gayi!

Kal interviewer poochega: *"Controlled component kya hota hai?"*

Tu confidently kahega: *"Input element jiska value React state se control hota hai. onChange event se state update hoti hai, aur state change hone pe input re-render hoti hai. Single source of truth — state."*

Woh impressed hoga. 100%.

Aaj ka Contact Form is portfolio-ready. Professional validation, great UX, clean code — Devsinc ke standards pe khara hai.

Commit karna mat bhoolna. Kal 7:30 AM pe fresh ho ke aana — Lists & Keys samajhenge, phir tera React foundation **rock solid** ho jayega!

Ab ja — validation logic perfect kar aur deploy kar! 🔥

Day 32 Complete — January 16, 2026 | MAX OMEGA PRIME Approved 